

МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
им. Н.Э. Баумана

Факультет “Информатика и системы управления”
Кафедра “Системы обработки информации и управления”



Дисциплина “Парадигмы и конструкции языков программирования”

Отчет по домашему заданию

Выполнил:
Студент группы ИУ5-34Б
Кубанов С.П.
Преподаватель:
Гапанюк Ю.Е.

Москва 2025

Задание

1. Выберите язык программирования (который Вы ранее не изучали) и (1) напишите по нему реферат с примерами кода или (2) реализуйте на нем небольшой проект (с детальным текстовым описанием).
2. Реферат (проект) может быть посвящен отдельному аспекту (аспектам) языка или содержать решение какой-либо задачи на этом языке.
3. Необходимо установить на свой компьютер компилятор (интерпретатор, транспилятор) этого языка и произвольную среду разработки.
4. В случае написания реферата необходимо разработать и откомпилировать примеры кода (или модифицировать стандартные примеры).
5. В случае создания проекта необходимо детально комментировать код.
6. При написании реферата (создании проекта) необходимо изучить и корректно использовать особенности парадигмы языка и основных конструкций данного языка.
7. Приветствуется написание черновика статьи по результатам выполнения ДЗ. Черновик статьи может быть подготовлен группой студентов, которые исследовали один и тот же аспект в нескольких языках или решили одинаковую задачу на нескольких языках.

Описание программы

Целью работы является освоение языка программирования **Kotlin** и инструментария **Android Studio**. В рамках задания реализован проект «Dog Clicker» — интерактивное приложение, в котором пользователь накапливает очки (клики), использует временные усиления (бусты) и настраивает параметры игры

Код MainActivity.kt

```
package com.example.project

// Импорт необходимых библиотек для Android, Compose и корутин
import android.content.Intent
import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.activity.enableEdgeToEdge
```

```

import androidx.compose.foundation.background
import androidx.compose.foundation.clickable
import androidx.compose.foundation.layout.*
import androidx.compose.material3.Scaffold
import androidx.compose.material3.Text
import androidx.compose.runtime.*
import androidx.compose.ui.res.colorResource
import androidx.compose.ui.tooling.preview.Preview
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import com.example.project.ui.theme.ProjectTheme
import androidx.compose.material3.Icon
import androidx.compose.runtime.saveable.rememberSaveable
import androidx.compose.ui.Modifier
import androidx.compose.ui.res.painterResource
import androidx.compose.foundation.border
import androidx.compose.ui.Alignment
import androidx.compose.ui.graphics.Color
import kotlinx.coroutines.delay
import kotlin.random.Random

// Главный класс активности приложения
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        // Установка содержимого экрана через Compose
        setContent {
            MyScreenPreview()
        }
    }
}

@Composable
private fun MyScreen() {
    // --- Инициализация состояний (State) ---

```

```

    // remember позволяет сохранить значение переменной при
    перерисовке (recomposition)

    // mutableStateOf делает переменную "наблюдаемой" — когда она
    меняется, экран обновляется

    var default by remember { mutableStateOf(2) } // Базовое значение
    клика

    var w by remember { mutableStateOf(2.dp) } // Ширина границы для
    кнопки Auto

    var ww by remember { mutableStateOf(2.dp) } // Ширина границы для
    кнопки Ratio

    var nclick by remember { mutableStateOf(true) } // Флаг: можно ли
    еще нажать на Ratio

    var pressed by remember { mutableStateOf(false) } // Состояние
    кнопки Auto (нажата/нет)

    // rememberSaveable сохраняет данные даже при повороте экрана

    var ratio by rememberSaveable { mutableIntStateOf(2) } // Текущий
    множитель клика

    var clicks by rememberSaveable { mutableIntStateOf(0) } // Общий
    счетчик кликов

    // Лямбда-функция для добавления очков

    val addClick: (Int) -> Unit = { x -> clicks = clicks + x }

    var boostTime by rememberSaveable { mutableIntStateOf(0) } //
    Таймер усиления x2

    var autoTime by rememberSaveable { mutableIntStateOf(0) } //
    Состояние автокликера

    // --- Побочные эффекты (Side Effects) ---

    // LaunchedEffect для таймера буста. Запускается при изменении
    boostTime

    LaunchedEffect(boostTime) {

        if (boostTime > 0) {

            delay(1000) // Задержка 1 секунда

            boostTime-- // Уменьшение времени

            if (boostTime == 0) {

```

```

        ratio = default // Возврат к обычному значению после
окончания буста
    }
}

// LaunchedEffect для автокликера. Работает в цикле, пока autoTime
> 0
LaunchedEffect(autoTime) {
    while (autoTime > 0) {
        delay(1000) // Ждем секунду
        addClick(ratio) // Автоматически добавляем текущий ratio к
счету
    }
}

// --- Верстка интерфейса ---
// Box — контейнер, позволяющий накладывать элементы друг на друга
или выравнивать их
Box(modifier = Modifier
    .fillMaxSize()
    .padding(horizontal = 50.dp, vertical = 30.dp)
){
    // Центральная кнопка (Нос собаки) для кликов
    Box(modifier = Modifier
        .width(60.dp)
        .align(Alignment.Center)
        .aspectRatio(1f)
        .background(color = colorResource(R.color.white))
        .clickable { addClick(ratio) } // При клике вызываем
функцию добавления
    ){
        Icon(
            painter = painterResource(R.drawable.dognose),
            contentDescription = null
        )
    }
}

```

```

// Кнопка усиления "x2" (Буст)
Box(modifier = Modifier
    .width(100.dp)
    .align(Alignment.BottomCenter)
    .aspectRatio(1f)
    .border(2.dp, Color.Black)
    .clickable {
        clicks = clicks - 50 // Списание стоимости (50 кликов)
        ratio = default * 2 // Удвоение множителя
        boostTime = 10      // Установка таймера на 10 секунд
    }
){
    Text(
        text = "  x2",
        fontSize = 50.sp,
        modifier = Modifier.align(Alignment.CenterStart)
    )
}

// Кнопка "Ratio" – установка случайного базового значения
Box(modifier = Modifier
    .width(100.dp)
    .align(Alignment.BottomStart)
    .aspectRatio(1f)
    .border(ww, Color.Black)
    .clickable {
        if (nclick) { // Сработает только один раз
            default = Random.nextInt(1, 10) // Случайное число
от 1 до 9

            ratio = default
            nclick = false
            ww = 5.dp // Визуальное изменение границы после
активации
        }
    }
}

```

```

){
    Text(
        text = "Ratio",
        fontSize = 30.sp,
        modifier = Modifier.align(Alignment.Center)
    )
}

// Кнопка "Auto" – включение/выключение автокликера
Box(modifier = Modifier
    .width(100.dp)
    .align(Alignment.BottomEnd)
    .border(w, Color.Black)
    .aspectRatio(1f)
    .clickable {
        if(pressed){
            ratio = default
            w = 2.dp
            pressed = false
            autoTime = 0 // Выключение
        } else {
            ratio = 1 // При автоклике множитель сбрасывается
            w = 5.dp
            pressed = true
            autoTime = 1 // Включение
        }
    }
)

Text(
    text = "  Auto",
    fontSize = 30.sp,
    modifier = Modifier.align(Alignment.CenterStart)
)
}

```

```

        // Отображение текущего счета
        Text(
            text = clicks.toString(),
            fontSize = 30.sp,
            modifier = Modifier.align(Alignment.TopEnd)
        )

        // Логика выбора картинки собаки (зависит от активного буста
x2)

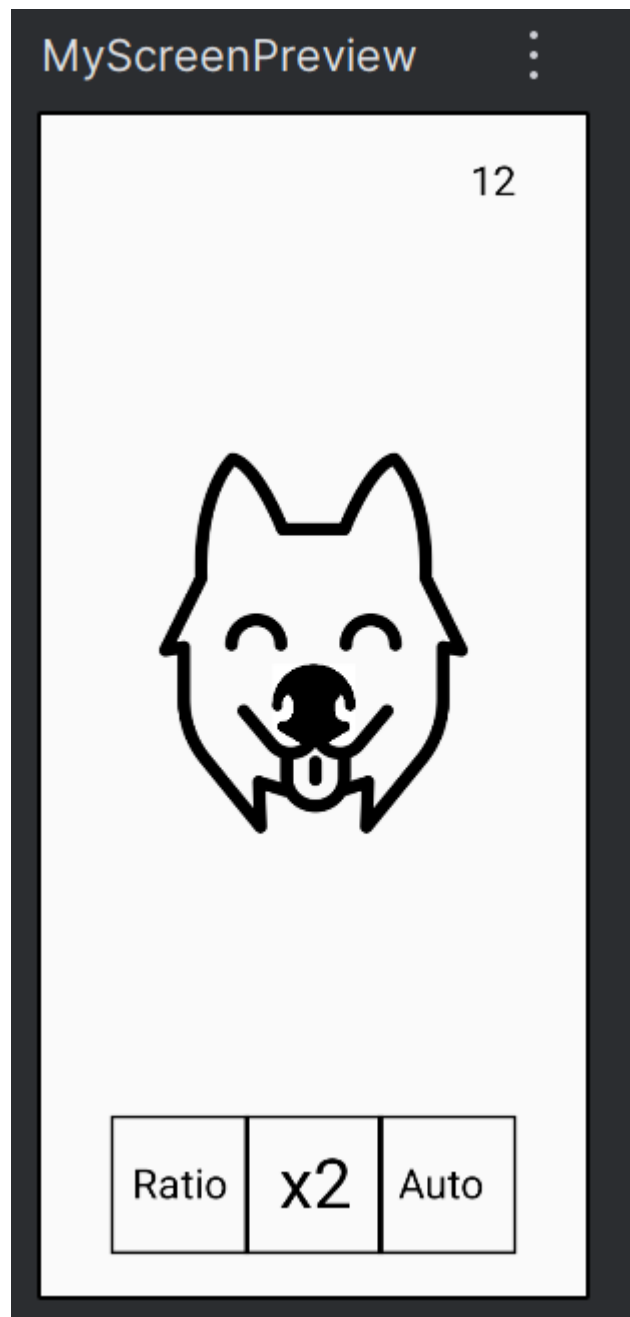
        val dogPainter = if (ratio == default * 2)
            painterResource(R.drawable.angrydog)
        else
            painterResource(R.drawable.dog)

        // Отображение иконки собаки с использованием смещения
(offset)
        Icon(
            painter = dogPainter,
            contentDescription = null,
            modifier = Modifier
                .size(300.dp)
                .offset(x = 0.dp, y = 200.dp)
        )
    }
}

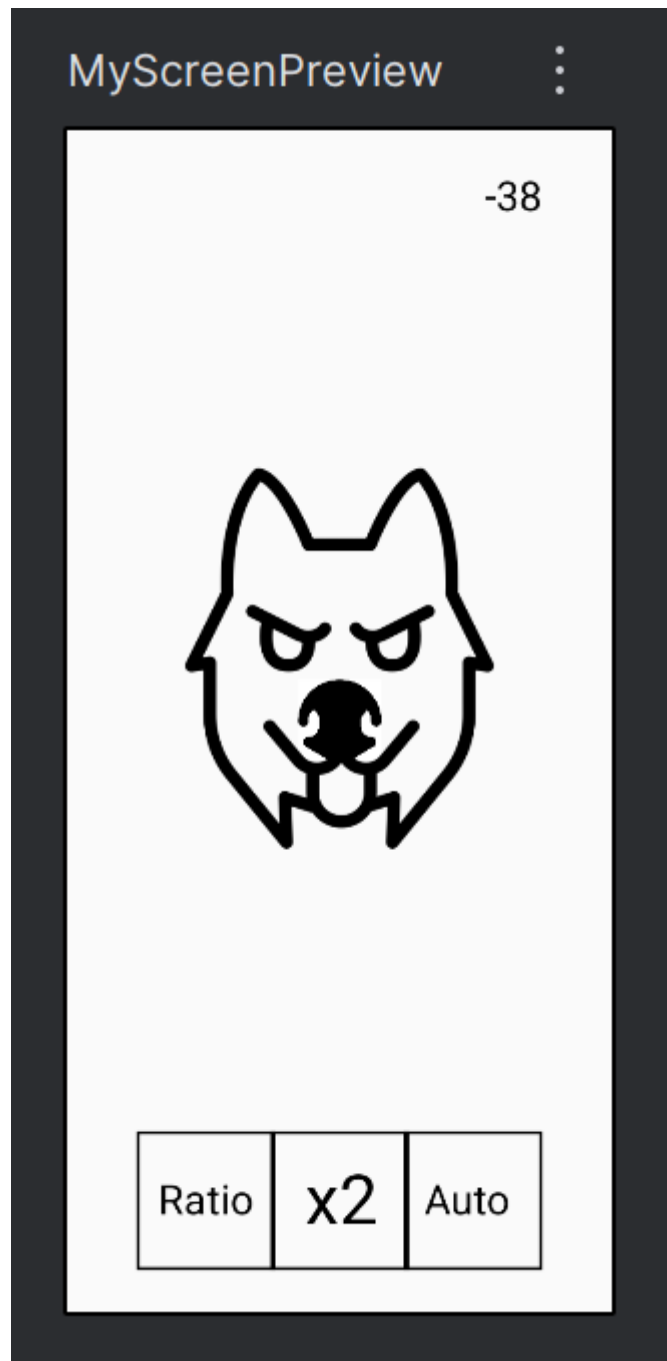
// Функция для предварительного просмотра в среде разработки
@Preview(showBackground = true)
@Composable
fun MyScreenPreview() {
    MyScreen()
}

```


Экраны



Главный экран. Кликаем, тогда добавляются очки в верхнем правом углу, по 2 за клик.



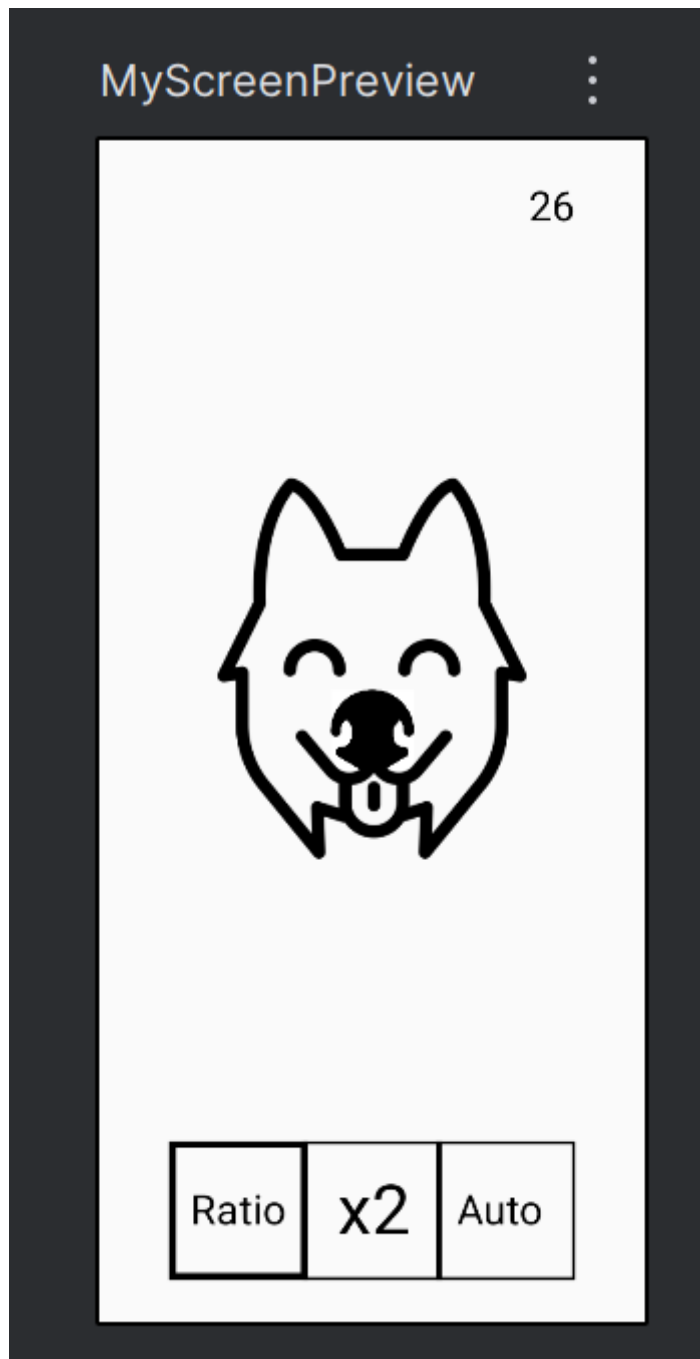
Включен режим x2: тратит 50 очков, но дает буст x2 на 10 секунд.



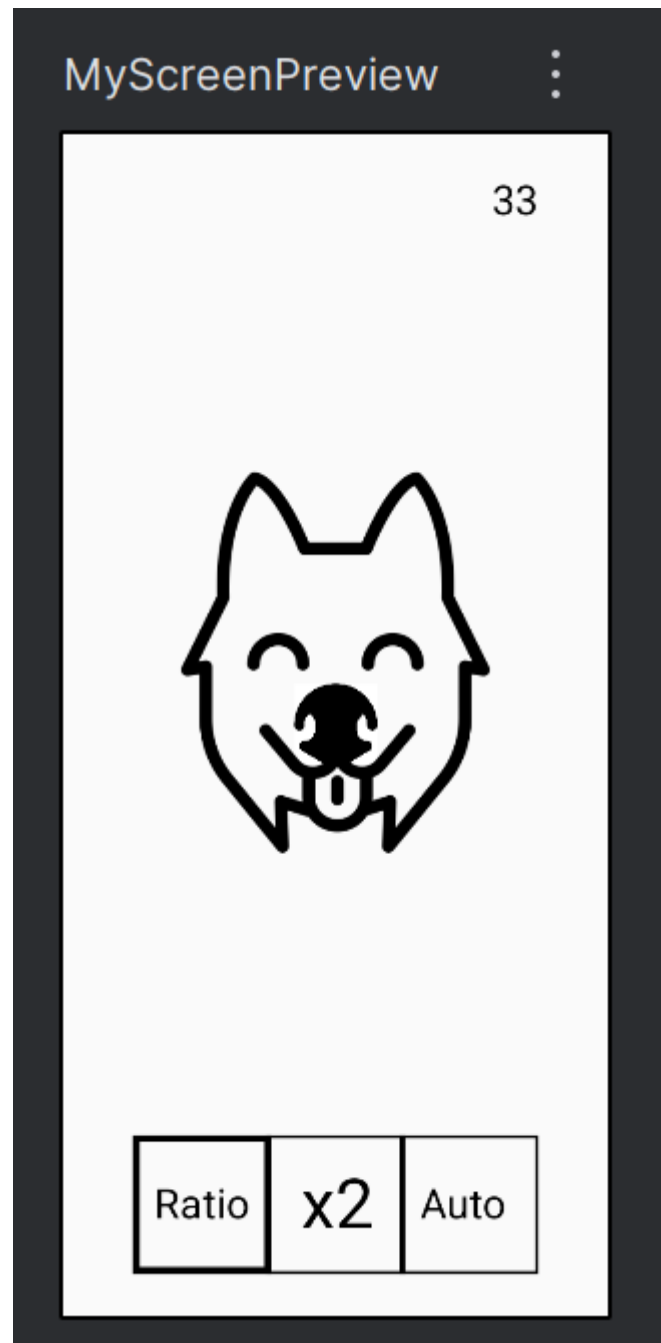
Включен режим x2. За клик не 2 очка, а 4.



Авторежим. Дается по 1 очку в секунду пока включен.



Данная кнопка позволяет выдать один раз случайное значение очков от 1 до 9, которое будет прибавляться за один клик.



Тут далось значение 7.